

Design Document: Smart Asset Management Platform

Project: Smart Asset Management and Resource Allocation Platform

Deliverable 2

1. Problem Understanding

Organizations, particularly campus bodies like the Cultural Council of IIT Roorkee, manage a large pool of shared resources (DSLR cameras, audio systems, stage props, etc.). Without a centralized system, resource allocation suffers from scheduling conflicts, manual tracking errors, delayed returns, and inefficient utilization. This platform addresses these issues by providing a full-stack, role-based asset management system. It digitizes the inventory, streamlines the booking and approval workflows, tracks real-time availability, and provides actionable analytics to administrators, ensuring transparent and efficient operational visibility.

2. System Architecture

The platform utilizes a modern Client-Server Architecture leveraging a Backend-as-a-Service (BaaS) for rapid development and real-time capabilities.

- **Frontend (Client):** Built using React and Vite, styled with Tailwind CSS and shadcn/ui components. It acts as a Single Page Application (SPA) offering two distinct role-based interfaces (User and Admin).
- **Backend & Database (BaaS):** Powered by Supabase. It provides a managed PostgreSQL database, instant RESTful APIs via PostgREST, and robust Authentication.
- **Security Layer:** Supabase Row Level Security (RLS) policies strictly enforce data access at the database level, ensuring users can only read/write authorized data.
- **Containerization:** Docker and Docker Compose are used to encapsulate the application for consistent local development and deployment.

3. Database Schema

The system relies on a relational schema comprising four core tables in PostgreSQL:

Table: users

- id: uuid (PK, references auth.users)
- email: varchar
- name: varchar
- role: enum ('admin', 'user')
- created_at: timestamp

Table: assets

- id: uuid (PK)
- name: varchar
- category: varchar
- description: text
- quantity_total: integer
- quantity_available: integer (Check constraint: >= 0)
- status: enum ('active', 'inactive')
- created_at: timestamp, updated_at: timestamp

Table: bookings

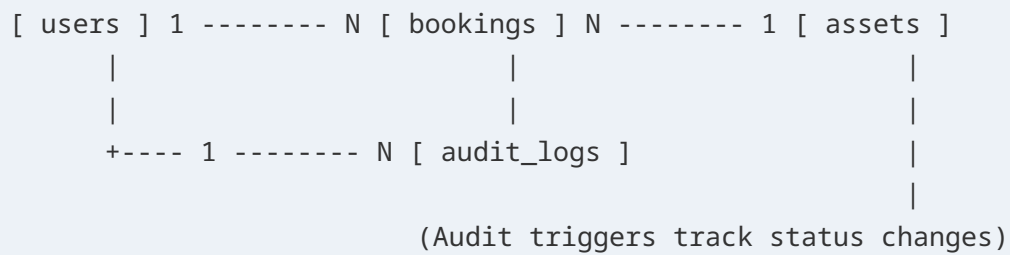
- id: uuid (PK)
- user_id: uuid (FK -> users.id)
- asset_id: uuid (FK -> assets.id)
- quantity_requested: integer
- start_date: date, end_date: date
- purpose: text
- status: enum ('pending', 'approved', 'rejected', 'issued', 'returned')
- admin_note: text
- created_at: timestamp, updated_at: timestamp

Table: audit_logs

- id: uuid (PK)
- user_id: uuid (FK -> users.id)
- action: varchar
- entity_type: varchar
- entity_id: uuid
- metadata: jsonb
- created_at: timestamp

4. Entity Relationship Diagram (ERD)

The following text-based representation maps the core relationships within the database. The users table has a one-to-many relationship with bookings and audit_logs. The assets table has a one-to-many relationship with bookings.



5. API Overview

The platform interfaces with the database via Supabase's auto-generated REST APIs and custom Remote Procedure Calls (RPCs) for complex atomic transactions.

Endpoint / Method	Description
GET /rest/v1/assets	Fetches the active inventory list. Filterable by category and search terms.
POST /rest/v1/bookings	Creates a new booking request. Requires user auth and valid asset constraints.
GET /rest/v1/bookings	Fetches booking histories. Users see their own; Admins see system-wide allocations.
RPC: approve_booking	A secure SQL transaction that atomically checks available quantity, decrements inventory, updates booking status to 'approved', and logs the audit event. Prevents race conditions.
PATCH /rest/v1/bookings	Used by admins to update states (e.g., 'issued', 'returned') and increment/decrement inventory accordingly.

6. Design Decisions

- **BaaS vs Custom Backend:** Chose Supabase to accelerate development, reducing boilerplate API code while providing enterprise-grade security via PostgreSQL Row Level Security (RLS).

- **Atomic Transactions for Inventory:** To prevent overbooking during concurrent requests, the approval workflow utilizes an SQL RPC (`approve_booking`) which locks the row, validates availability, and updates the count atomically.
- **Role-Based Access Control (RBAC):** Managed natively via RLS. A user's role is determined by their UUID in the auth context, ensuring unauthorized users cannot bypass UI restrictions via direct API calls.
- **Audit Logging:** Implemented a centralized `audit_logs` table to track all inventory state changes, providing accountability and ensuring a transparent paper trail for the Cultural Council's valuable assets.
- **Modular UI Architecture:** Adopted `shadcn/ui` to ensure consistent, accessible, and responsive components, allowing rapid assembly of complex dashboard charts (via `Recharts`) and data tables.